

# Assignment 5

## 1. Main idea of autoregressive models

Autoregressive models generate data one element at a time by predicting the next value based on previous values.

They assume that any data can be represented as a sequence

This means the model predicts each token or pixel conditioned on everything generated before it.

They model the full probability distribution using the **chain rule of probability**

**The important point is that autoregressive models compute exact likelihoods  $P(X)$**

Unlike:

- VAEs → Approximates  $P(X)$  using latent variables and ELBO
- GANs → use adversarial training Does not compute  $P(X)$  directly
- Diffusion models → Learns reverse denoising process instead of direct  $P(X)$

## 2. How are autoregressive models trained?

During training, the correct previous tokens are provided to the model, and it learns to predict the next token.

This is standard supervised learning because:

- input = previous tokens
- target = next token
- the model minimizes prediction error
- training uses normal optimization methods like gradient descent
- The loss function is usually **cross-entropy loss** (negative log-likelihood).

Autoregressive training is more stable than Generative Adversarial Network training because:

- there is only one network learning/ a direct loss function like cross-entropy
- no competition between generator and discriminator
- no adversarial instability
- no mode collapse

### 3. Why is pixel-by-pixel generation expensive?

In models like PixelCNN, every pixel must be generated sequentially. For high-resolution images, this means predicting millions of pixels one at a time. Each new pixel depends on all previous pixels, preventing parallel generation. This makes generation extremely slow and computationally expensive.

### 4. Why compress images into tokens?

Modern models first compress images into smaller latent tokens using encoders or VAEs.

Process:

1. Image encoder compresses image
2. Image becomes a grid of tokens
3. Autoregressive model generates tokens
4. Decoder reconstructs full image

This improves efficiency because:

- the sequence becomes much shorter
- computation and memory usage decrease
- generation becomes faster while still preserving important image features

### 5. Why are transformers preferred over RNNs?

because they solve many limitations of recurrent models.

1. Transformers process sequences in parallel during training, making them much faster than RNNs.
2. They also handle long-range dependencies better using self-attention.
3. Transformers scale better to large datasets and produce higher-quality results.
4. They also use **causal masking**

## 6. Compare autoregressive and diffusion models

### Generation process

- **Autoregressive models:** generate one token/pixel at a time sequentially.
- **Diffusion models:** start from noise and iteratively denoise the image until a clean sample appears. .

### Which is faster?

Autoregressive models are usually faster for each generation step because they directly predict the next token.

However, they are still sequential.

Diffusion models often require many denoising steps, making generation slower overall. That is why diffusion sampling can take much longer than autoregressive generation.